

Computation Graph and Forward Propagation

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

Course: Deep Learning and Neural Network

Course Code: CS4152

November 6, 2025

Motivation

- Neural network computation is organized in two phases:
 - **Forward pass**: compute outputs.
 - **Backward pass**: compute gradients (derivatives).
- A **computation graph** makes dependencies explicit and guides backpropagation.

Target Function

We want to compute

$$J = 3(a + b c)$$

with inputs a, b, c .

Step-by-Step Computation

Step	Operation	Expression	Output
1	Multiply	$u = b c$	u
2	Add	$v = a + u$	v
3	Scale	$J = 3v$	J

Numerical Example

If $a = 5$, $b = 3$, $c = 2$:

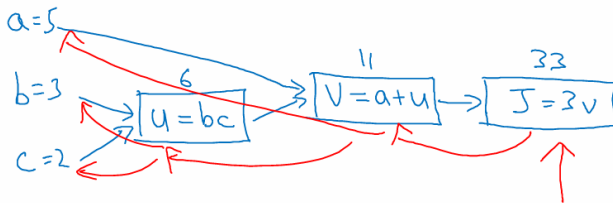
$$u = bc = 6, \quad v = a + u = 11, \quad J = 3v = 33.$$

Check: $J = 3(5 + 3 \cdot 2) = 33$.

Computation Graph

$$J(a,b,c) = 3(a + \underbrace{bc}_u) = 3(\underbrace{5 + 3 \times 2}_v) = 33$$

$$\begin{aligned} u &= bc \\ v &= a + u \\ J &= 3v \end{aligned}$$



Why Computation Graphs?

- Clarify data flow and **dependencies** between variables.
- Enable modular forward computation and systematic backward gradients.
- Foundation for **backpropagation** in deep networks.

Summary

- Forward pass (left to right) computes J .
- Backward pass (right to left) will compute $\frac{\partial J}{\partial a}, \frac{\partial J}{\partial b}, \frac{\partial J}{\partial c}$.
- Next: **reverse-mode** (backprop) on this graph.